

Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт
Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №1
Системы поддержки принятия решений
тема

Преподаватель

подпись, дата

С.С. Булыкин
инициалы, фамилия

Студент КИ23-17/1Б, 032323896
номер группы, зачетной книжки

подпись, дата

Ф.В. Фесенко
инициалы, фамилия

Красноярск 2026

1 Цель

Знакомство с архитектурой хранилищ данных и технологиями OLAP-анализа, формирование навыков проектирования и наполнения хранилища данных, а также выполнения OLAP-анализа.

2 Задачи

1. Спроектировать и наполнить БД
2. Спроектировать архитектуру хранилища данных
3. Создать хранилище данных
4. Наполнить хранилище данных
5. Составить перечень аналитических задач
6. Спроектировать систему аналитической отчетности

3 Ход работы

В качестве предметной области был выбран сервис такси. Моделируются данные о поездках: кто вез, откуда, в каком городе, когда, за какую цену и на какой машине.

В качестве OLTP-источника выбран PostgreSQL — реляционная СУБД, обеспечивающая ACID-транзакции и целостность данных. База содержит 5 нормализованных таблиц: regions, cities, drivers, cars, trips — с 5 регионами, 20 городами, 200 водителями, 200 машинами и 50 000 поездок за 2023–2024 годы.

В качестве СУБД для хранилища выбран ClickHouse — колоночная аналитическая СУБД. Колоночное хранение позволяет аналитическому запросу читать только те колонки, которые нужны, а не всю таблицу целиком, что даёт ускорение агрегатов в десятки раз по сравнению с строчными СУБД.

На уровне способа OLAP выбран ROLAP: многомерный куб не хранится физически, а эмулируется SQL-запросами поверх реляционных таблиц ClickHouse. Все OLAP-операции — slice, dice, roll-up, drill-down, pivot — выполняются обычным SQL без отдельного OLAP-сервера.

В качестве ETL-инструмента выбран Python, сначала происходит чтение таблиц PostgreSQL в Pandas, затем происходит преобразование OLTP в модель

“звезда”: денормализация, вычисления необходимых измерений, далее данные вставляются в ClickHouse. В качестве BI-платформы выбран Superset, так как поддерживает работу ClickHouse из коробки. Весь стек развёрнут в Docker-контейнерах через Docker Compose, что обеспечивает изоляцию и воспроизводимость результата.

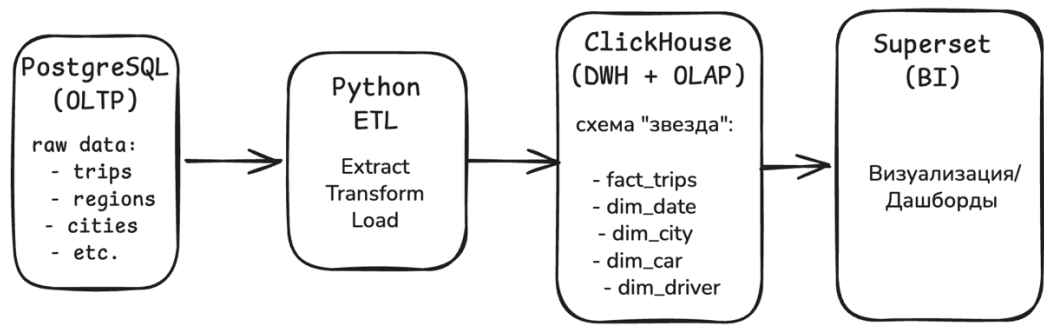


Рисунок 1 – Схема потока данных

Пример данных из таблицы поездок представлен на рисунке 2.

```
taxi_oltp=# select * from trips limit 10;
```

trip_id	driver_id	car_id	city_id	started_at	duration_min	distance_km	price	status
1	1	1	1	2023-09-28 11:44:33.102574	19	21.16	782.53	completed
2	2	2	2	2024-06-20 03:52:36.156243	23	4.41	248.95	completed
3	3	3	3	2024-05-20 12:57:55.724991	51	20.32	203.59	completed
4	4	4	4	2023-04-15 04:07:25.36525	45	27.46	410.14	completed
5	5	5	5	2024-01-10 18:23:11.602426	57	19.32	1391.56	completed
6	6	6	6	2023-07-26 23:53:08.717082	27	2.12	452.98	completed
7	7	7	7	2024-03-01 13:07:14.957466	54	7.71	1443.14	completed
8	8	8	8	2024-06-20 02:40:35.87397	20	25.92	1305.92	completed
9	9	9	9	2023-03-27 23:54:42.699775	30	11.44	1521.06	completed
10	10	10	10	2023-03-24 00:38:21.106367	34	24.71	1415.00	completed

(10 rows)

Рисунок 2 – Таблица поездок (OLTP)

Схема хранилища данных (ROLAP) представлена на рисунке 3.

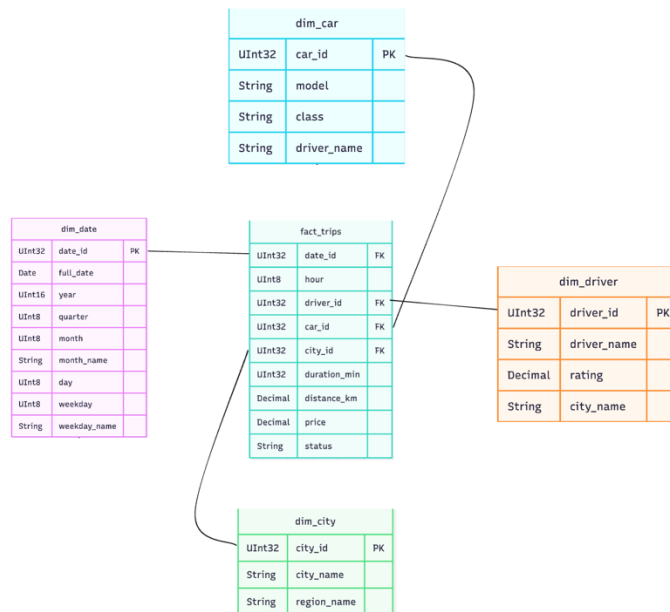


Рисунок 3 – Схема ROLAP

ETL был реализован с помощью Python и библиотек: Pandas, clickhouse-connect и других.

Далее приведены примеры OLAP-операций. На рисунке 4 представлена операция среза по одному измерению (2024 год) и результат ее работы.

```
dce3ff55de8c :) SELECT c.city_name, sum(f.price) AS revenue
FROM dwh.fact_trips f
JOIN dwh.dim_city c ON c.city_id = f.city_id
JOIN dwh.dim_date d ON d.date_id = f.date_id
WHERE d.year = 2024 AND f.status = 'completed'
GROUP BY c.city_name
ORDER BY revenue DESC;
```

```
SELECT
    c.city_name,
    sum(f.price) AS revenue
FROM dwh.fact_trips AS f
INNER JOIN dwh.dim_city AS c ON c.city_id = f.city_id
INNER JOIN dwh.dim_date AS d ON d.date_id = f.date_id
WHERE (d.year = 2024) AND (f.status = 'completed')
GROUP BY c.city_name
ORDER BY revenue DESC
```

Query id: 609eb5a9-d038-4fe2-8d73-8158313bbc7a

	city_name	revenue
1.	Город-11	1073900.19
2.	Город-8	1045510.32
3.	Город-16	1041779.12
4.	Город-19	1040198.89
5.	Город-7	1037043.81
6.	Город-12	1030005.15
7.	Город-3	1029195.16
8.	Город-20	1026819.37
9.	Город-10	1025966.29
10.	Город-5	1020051.93
11.	Город-15	1019929.01
12.	Город-14	1018384.01
13.	Город-9	1017675.49
14.	Город-13	1017416.22
15.	Город-1	1007641.66
16.	Город-6	1003890.93
17.	Город-18	996525.3
18.	Город-17	987831.95
19.	Город-2	982026.1
20.	Город-4	978162.18

20 rows in set. Elapsed: 0.085 sec. Processed 50.75 thousand rows, 1.70 MB (598.65 thousand rows/s., 20.11 MB/s.)
Peak memory usage: 105.81 KiB.

Рисунок 4 – Операция среза

На рисунке 5 представлен пример операции Dice (нарезки по нескольким параметрам: 2024 год + регион Сибирь).

```
dce3ff55de8c :) SELECT c.city_name, sum(f.price) AS revenue
FROM dwh.fact_trips f
JOIN dwh.dim_city c ON c.city_id = f.city_id
JOIN dwh.dim_date d ON d.date_id = f.date_id
WHERE d.year = 2024 AND c.region_name = 'Сибирь'
GROUP BY c.city_name;
```

```
SELECT
    c.city_name,
    sum(f.price) AS revenue
FROM dwh.fact_trips AS f
INNER JOIN dwh.dim_city AS c ON c.city_id = f.city_id
INNER JOIN dwh.dim_date AS d ON d.date_id = f.date_id
WHERE (d.year = 2024) AND (c.region_name = 'Сибирь')
GROUP BY c.city_name
```

Query id: e4adaa31-5116-4cca-9b7d-db436364641c

	city_name	revenue
1.	Город-3	1138083.35
2.	Город-1	1123345.1
3.	Город-4	1088879.86
4.	Город-2	1099465.36

4 rows in set. Elapsed: 0.044 sec. Processed 50.75 thousand rows, 805.34 KB (1.16 million rows/s., 18.43 MB/s.)
Peak memory usage: 95.39 KiB.

Рисунок 5 – Пример операции Dice

На рисунке 6 представлен пример операции Roll-up (свертка год, квартал, месяц).

```
dce3ff55de8c :) SELECT d.year, d.quarter, d.month, sum(f.price) AS revenue
FROM dwh.fact_trips f
JOIN dwh.dim_date d ON d.date_id = f.date_id
GROUP BY ROLLUP(d.year, d.quarter, d.month) limit 10;

SELECT
    d.year,
    d.quarter,
    d.month,
    sum(f.price) AS revenue
FROM dwh.fact_trips AS f
INNER JOIN dwh.dim_date AS d ON d.date_id = f.date_id
GROUP BY
    d.year,
    d.quarter,
    d.month
WITH ROLLUP
LIMIT 10

Query id: 52c5f581-037f-4fea-83f7-2e903f9f291c
```

	year	quarter	month	revenue
1.	2024	1	3	1923313.35
2.	2024	3	9	1819157.36
3.	2023	2	6	1869642.43
4.	2023	4	11	1829413.32
5.	2024	2	5	1951230.07
6.	2023	2	4	1841698.75
7.	2024	4	10	1921296.02
8.	2024	1	1	1976525.69
9.	2023	4	12	1914004.98
10.	2023	1	2	1744817.06

10 rows in set. Elapsed: 0.030 sec. Processed 50.73 thousand rows, 605.87 KB (1.69 million rows/s., 20.17 MB/s.)
Peak memory usage: 199.69 KiB.

Рисунок 6 – Пример операции Roll-up

На рисунке 7 представлен пример операции pivot – поворот (годы в колонки).

```
dce3ff55de8c :) SELECT
    car.class,
    sumIf(f.price, d.year = 2023) AS y2023,
    sumIf(f.price, d.year = 2024) AS y2024
FROM dwh.fact_trips f
JOIN dwh.dim_car car ON car.car_id = f.car_id
JOIN dwh.dim_date d ON d.date_id = f.date_id
GROUP BY car.class;

SELECT
    car.class,
    sumIf(f.price, d.year = 2023) AS y2023,
    sumIf(f.price, d.year = 2024) AS y2024
FROM dwh.fact_trips AS f
INNER JOIN dwh.dim_car AS car ON car.car_id = f.car_id
INNER JOIN dwh.dim_date AS d ON d.date_id = f.date_id
GROUP BY car.class

Query id: 1b62d777-1cea-48eb-8ffd-b085445bdb89
```

	class	y2023	y2024
1.	business	5338493.94	5457981.39
2.	economy	6033431.65	6086748.99
3.	comfort	10970552.95	11115075.93

3 rows in set. Elapsed: 0.019 sec. Processed 50.93 thousand rows, 808.46 KB (2.62 million rows/s., 41.61 MB/s.)
Peak memory usage: 238.58 KiB.

Рисунок 7 – Пример операции pivot

Далее с помощью инструмента Apache Superset был создан датасет – плоская таблица, объединяющая факт поездки со всеми измерениями с помощью join-ов. Это позволяет визуализировать метрики быстро.

В ходе работы были сформулированы и решены следующие аналитические задачи:

1. Какова общая выручка сервиса такси за анализируемый период?
2. Как меняется выручка по месяцам?
3. Как меняется средний чек по месяцам?
4. Как меняется среднее расстояние поездки по месяцам?
5. Какие города приносят наибольшую выручку?
6. В каких городах чаще всего отменяют поездки?
7. В какие часы суток наибольший спрос на поездки?
8. Какова доля отмененных поездок относительно завершенных?

Для решение данных задач было сформировано 3 дашборда. Они представлены на рисунке 8-10.

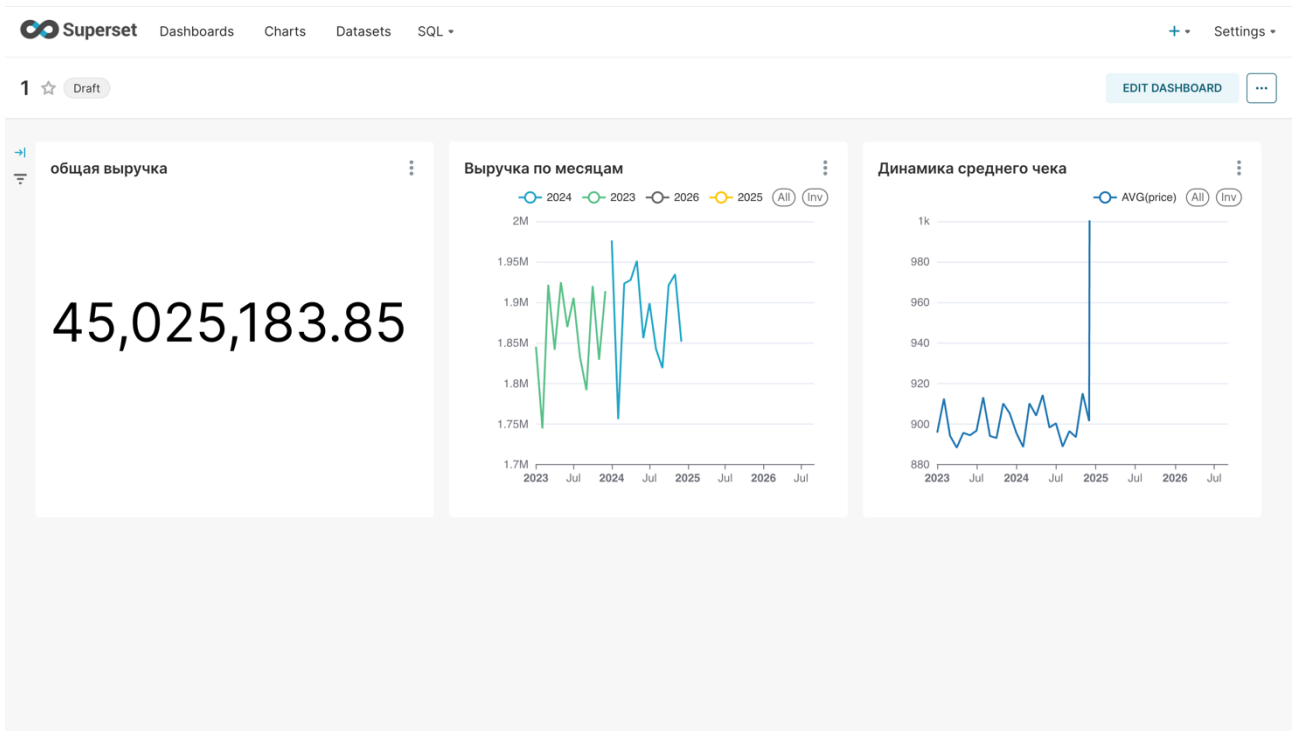


Рисунок 8 – Дашборд 1

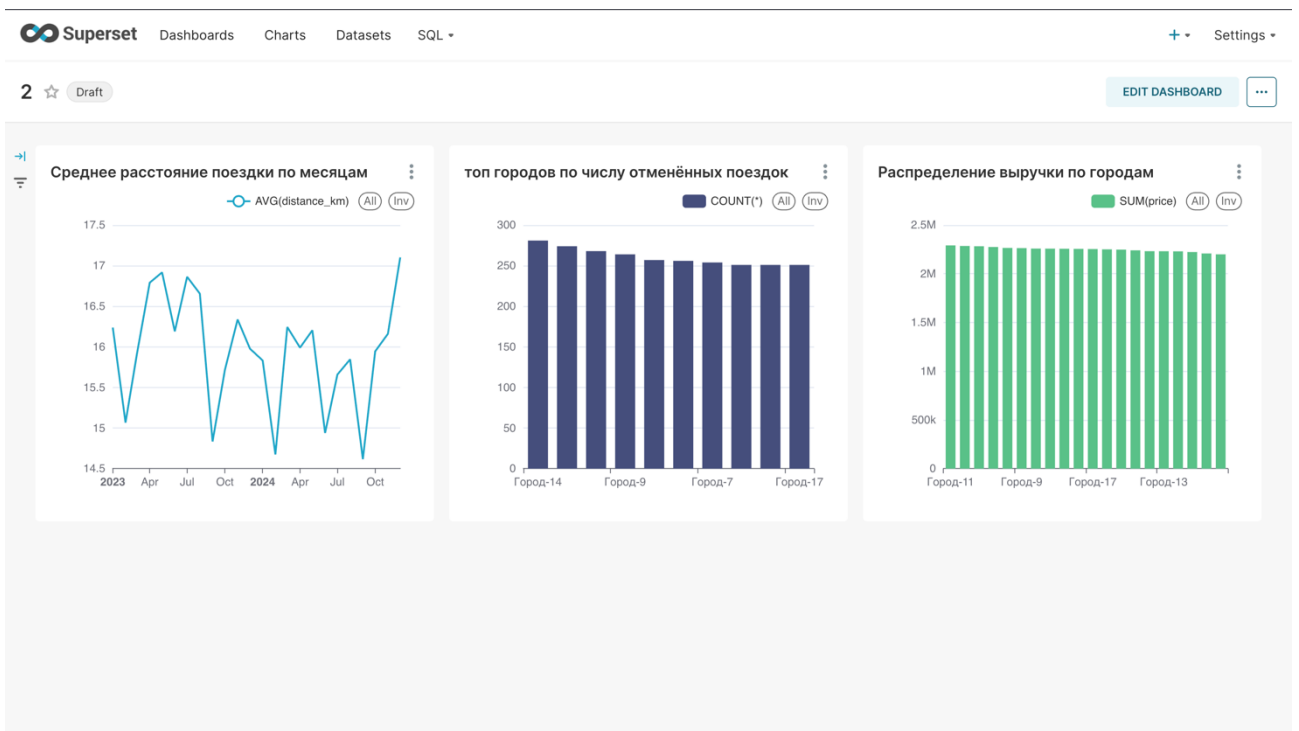


Рисунок 9 – Дашборд 2

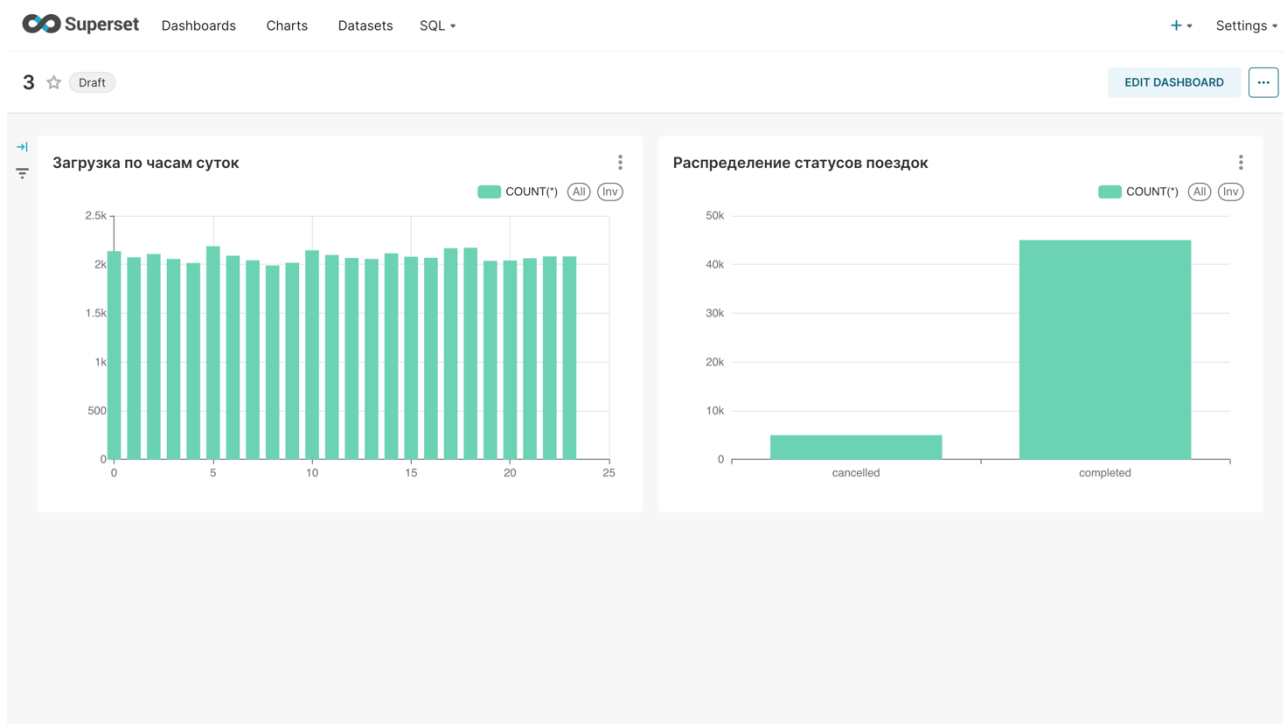


Рисунок 10 – Дашборд 3

4 Выводы

В ходе практической работы спроектирована и реализована система поддержки принятия решений для сервиса такси — от OLTP-источника до интерактивной отчётности. В PostgreSQL хранятся пять нормализованных таблиц с 50 000 сгенерированных поездок, в ClickHouse создано хранилище по схеме «звезда» (факт `fact_trips` + четыре измерения). Выбран ROLAP-подход, при котором многомерный куб эмулируется SQL-запросами без отдельного OLAP-сервера. ETL реализован на Python (pandas, SQLAlchemy, clickhouse-connect), выполнены все основные OLAP-операции (slice, dice, roll-up, drill-down, pivot), а на Apache Superset построен дашборд из восьми чартов с фильтрами.

В результате решены все поставленные задачи: спроектирован источник, создано хранилище, реализован ETL, выполнен OLAP-анализ и построена система отчётности. Весь стек контейнеризован через Docker Compose, что обеспечивает воспроизводимость. Освоены принципы проектирования хранилищ, различия OLTP и OLAP, схемы «звезда» и «снежинка», ETL и ROLAP.